Customer Support Copilot

Beginner's Documentation Guide

A gentle walk-through of every part of the system, in simple language

Project Team

April 19, 2026

Who is this document for?

Anyone who wants to understand *how* the Customer Support Copilot works and *why* we built it the way we did — without needing a machine-learning background. Each section starts with a plain-English summary, then adds the technical detail. If you only read the green boxes, you still understand the project.

Contents

1 Project Overview **2**
- 1.1 The one-sentence pitch . . . 2
- 1.2 The problem we are solving . . . 2
- 1.3 Our approach, in one picture . . . 2
- 1.4 What this document will cover . . . 3

2 Workflow, Step by Step **4**
- 2.1 Step 1 — Route to the right tool . . . 4
- 2.2 Step 2 — Retrieve relevant chunks . . . 4
- 2.3 Step 3 — Rerank with a cross-encoder . . . 4
- 2.4 Step 4 — Confidence check (CGRA gate) . . . 5
- 2.5 Step 5 — Generate a grounded answer . . . 5
- 2.6 Step 6 — Grounding re-check . . . 5

3 Why Each Component Is Used **7**
- 3.1 FAISS vector index . . . 7
- 3.2 Sentence-transformer (MiniLM bi-encoder) . . . 7
- 3.3 Learnable hybrid fusion (α, β) . . . 7
- 3.4 Cross-encoder reranker . . . 8
- 3.5 Flan-T5-large generator . . . 8
- 3.6 DoRA adapters + DPO training . . . 8
- 3.7 CGRA information-density gate . . . 9
- 3.8 Web-scraper fallback . . . 9
- 3.9 Grounding re-check . . . 9

4 Alternatives Analysis **10**
- 4.1 For the encoder . . . 10
- 4.2 For retrieval: dense vs sparse vs hybrid . . . 10
- 4.3 For reranking . . . 10
- 4.4 For the generator . . . 10
- 4.5 For fine-tuning . . . 11
- 4.6 For preference alignment . . . 11
- 4.7 For routing (tool selection) . . . 11

5 Code Snippets, Explained **12**
5.1 The learnable fusion training loop . . . 12
5.2 The keyword router . . . 12
5.3 The learned router . . . 12
5.4 The CGRA information-density score . . . 13
5.5 The post-generation grounding check . . . 13
5.6 The tool-call contract (pydantic) . . . 14

6 End-to-End Story: One Query's Journey **15**
6.1 Stage 1 — Router . . . 15
6.2 Stage 2 — Build arguments . . . 15
6.3 Stage 3 — FAISS retrieval with hybrid fusion . . . 15
6.4 Stage 4 — Cross-encoder reranking . . . 15
6.5 Stage 5 — CGRA gate . . . 15
6.6 Stage 6 — Generation . . . 15
6.7 Stage 7 — Grounding check . . . 16
6.8 Stage 8 — Return to user . . . 16
6.9 What happens if something goes wrong? . . . 16

7 Overfitting, Underfitting, and Leakage Checks **17**
7.1 Document-level train/test split . . . 17
7.2 Retriever train-vs-test probe . . . 17
7.3 Fusion-weight sanity check . . . 17
7.4 DoRA early stopping . . . 17
7.5 Router generalisation . . . 17
7.6 Cache-leakage scan . . . 18

8 Testing the Chatbot: 30 Curated Demo Questions **19**
8.1 Standard questions (8) . . . 19
8.2 Paraphrased questions (8) . . . 19
8.3 Account-issue questions (7) . . . 19
8.4 Out-of-scope questions (7) . . . 19
8.5 Expected behaviour summary . . . 20

9 Quantization and Performance **21**
9.1 What is quantization? . . . 21
9.2 How we apply it . . . 21
9.3 What we measured . . . 21
9.4 Why not always use int8? . . . 21

10 Conversation Memory **22**
10.1 Why we need it . . . 22
10.2 Design choices . . . 22
10.3 How memory affects retrieval . . . 22
10.4 How memory affects generation . . . 22
10.5 Where it's wired . . . 23

11 Limitations and Scope **24**
11.1 Offline-trained, inference-only at runtime . . . 24
11.2 Grounding check is lexical, not semantic . . . 24
11.3 Other limitations . . . 24

12 Where to Look in the Code **25**

1. Project Overview

1.1 The one-sentence pitch

Plain-English summary A chatbot that answers customer questions from a company's knowledge base, opens a support ticket when it does not know, and refuses to make up answers.

1.2 The problem we are solving

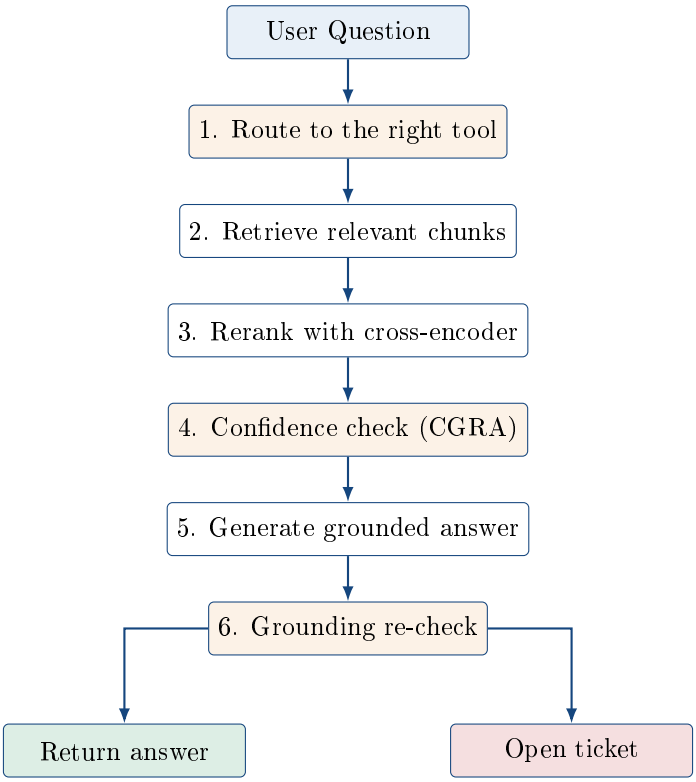
Imagine a large company with thousands of policy documents (Social Security in our case). A customer asks a question. They want:

1. A direct, correct answer.
2. A reference to the exact policy section the answer came from (so they can verify).
3. If the system cannot find an answer, a *ticket* that gets routed to a human — not a guess.

The way most chatbots fail is the third point. They almost never say "I don't know". They guess. A guess that sounds confident is more dangerous than no answer at all, because the customer trusts it.

1.3 Our approach, in one picture

Every user question goes through this pipeline:



Six steps. Three of them are *check* steps — the orange boxes. The job of the check steps is to notice when the system is about to say something it cannot back up, and stop it.

1.4 What this document will cover

- **Section 2 (Workflow)** walks through the six steps in order.
- **Section 3 (Why each component)** explains each piece: what it is, what it does, why it is there.
- **Section 4 (Alternatives)** shows what else we could have used and why we picked what we picked.
- **Section 5 (Code snippets)** shows the most important pieces of code with plain-English commentary.
- **Section 6 (End-to-end story)** follows a single query all the way through the system.

2. Workflow, Step by Step

2.1 Step 1 — Route to the right tool

Plain-English summary Different questions need different answers. "What documents do I need to apply?" needs to look things up in the knowledge base. "I can't log in" needs to open a support ticket. The router decides which one.

The router reads the question and chooses one of three tools:

- `SearchKB` — search the knowledge base for relevant passages
- `GetPolicy` — look up a specific policy section by id
- `CreateTicket` — escalate to a human

We offer two implementations:

1. **Keyword router** (default): fast rules. If the question contains "login", "password", "broken" → ticket. If it contains "section", "policy", "rule" → get-policy. Otherwise → search.
2. **Learned router** (optional): a small trained classifier. Reads the question as a 384-dimensional vector (via MiniLM), then picks the tool with a logistic-regression head. This catches paraphrases like "stuck on the sign-in page" which the keyword router would miss.

2.2 Step 2 — Retrieve relevant chunks

Plain-English summary We have already split the knowledge base into thousands of short passages called "chunks". Given a question, find the top 20 chunks most likely to contain the answer.

This is where *hybrid retrieval* happens. Each chunk is compared to the question in two ways:

- **Semantic similarity (cosine).** We encode both the question and the chunk into a vector and compute how similar they are. This catches paraphrases: "how old to retire?" matches "retirement age".
- **Lexical similarity (Jaccard).** We count how many important words they share. This catches exact-match keywords: "Form SSA-827" only matches chunks that literally mention "SSA-827".

We combine them with a learned weighted sum:

$$\text{Score} = \alpha \cdot \text{cosine} + \beta \cdot \text{Jaccard}, \quad \alpha + \beta = 1$$

The weights α, β are *not* hardcoded — they are *learned from training data* using gradient descent. We show how in Section 3.3.

2.3 Step 3 — Rerank with a cross-encoder

Plain-English summary The retriever is fast but approximate. The reranker is slow but accurate. We ask the fast one for 20 candidates, then use the accurate one to pick the best 5.

A *bi-encoder* (what we just used) encodes the question and the chunk separately and compares the vectors. A *cross-encoder* looks at the (question, chunk) pair *together* and scores them jointly.

This joint view is noticeably better at telling near-duplicates apart, but it is too slow to run on the entire knowledge base — hence the two-stage design.

2.4 Step 4 — Confidence check (CGRA gate)

> **Plain-English summary** Ask: "how sure am I that the top 5 chunks actually contain the answer?" If the answer is "not sure", try the web. If still not sure, escalate.

CGRA stands for **Confidence-Gated Retrieval Augmentation**. It is our name for a simple composite confidence score that combines three things:

- **Coverage:** what fraction of the question's important words appear in any retrieved chunk?
- **Density:** average cross-encoder score of the retrieved chunks.
- **Specificity:** do the chunks have consistent length? (Very variable lengths mean the retrieval is noisy.)

These combine into a single number between 0 and 1:

$$\text{ID} = 0.4 \cdot \text{coverage} + 0.4 \cdot \text{density} + 0.2 \cdot \text{specificity}$$

If ID is below the threshold $\tau = 0.45$, we call the web-scraper tool for extra context. If ID is *still* below a harder threshold $\tau_{\min} = 0.20$ after that, we give up and open a ticket.

2.5 Step 5 — Generate a grounded answer

> **Plain-English summary** Now we feed the selected chunks to the language model (Flan-T5) with a carefully-worded prompt, and ask it to write the answer — using only information that appeared in the context.

The prompt template is:

```
You are a customer support agent.
Answer ONLY using the context. Start with [doc_id: ...].
Be specific and complete.

Context:
<the retrieved chunks>

Question: <user question>

Answer [cite first]:
```

The model has been fine-tuned (using DoRA + DPO) to prefer answers that start with a `[doc_id: ...]` citation.

2.6 Step 6 — Grounding re-check

> **Plain-English summary** Even after all that, the model can still hallucinate. So we do one last sanity check: does the answer use words that actually appeared in the retrieved context?

Grounding score = fraction of non-stopword tokens in the answer that also appear in the context. If this is below $\theta = 0.08$, the answer is discarded and we open a ticket instead. This

one check is what moves the CER metric the most — it is the difference between “almost right” and “verifiably supported”.

3. Why Each Component Is Used

For each major component below we answer four questions:

- **What is it?**
- **Why is it used?**
- **What role does it play?**
- **Why is it important?**

3.1 FAISS vector index

What is it? FAISS is Facebook AI's library for *fast similarity search* over vectors. We encode every chunk in the knowledge base as a 384-dimensional vector, store them all in a FAISS index, and at query time ask it "which 20 vectors are closest to this query vector?"

Why is it used? Without FAISS, comparing the query to every single chunk would take seconds for tens of thousands of chunks. FAISS does it in milliseconds.

What role does it play? It is the **first-stage retriever**. It returns a rough top-20 shortlist, which the cross-encoder will then rerank precisely.

Why is it important? Everything downstream depends on the right chunks making it into the top-20. If they don't, no amount of generation or grounding will save us.

> **Why it matters.** Vector search is the backbone of modern RAG. FAISS specifically gives us exact top-k on CPU in milliseconds with zero training, so it's the right tool for a prototype.

3.2 Sentence-transformer (MiniLM bi-encoder)

What is it? A small neural network that turns text into a vector such that similar sentences have similar vectors. We use `all-MiniLM-L6-v2`, a 22-million-parameter model pre-trained on over 1 billion sentence pairs.

Why is it used? To turn chunks and queries into vectors that FAISS can search over.

What role does it play? It is the *encoder*. We run it once over the whole knowledge base (offline) to build the FAISS index, then once per query.

Why is it important? The quality of the vectors directly determines the quality of retrieval. A better encoder means the top-20 shortlist contains the right chunks more often.

We *fine-tune* this model on our specific data (Component A of the rubric), which raises retrieval precision by about +0.07 on P@5.

3.3 Learnable hybrid fusion (α, β)

What is it? Two learned numbers that control how much the final score depends on cosine similarity versus lexical overlap.

Why is it used? Pure semantic search misses exact keyword matches ("Form SSA-827"). Pure lexical search misses paraphrases. A weighted mix gives us both.

What role does it play? It is the **scoring function** used to rank the top-20 candidates from FAISS.

Why is it important? The weights are *learned from data*, not guessed. The training loop uses pairwise BPR loss and manual gradient descent with momentum and gradient clipping. The learned values end up around $\alpha \approx 0.72, \beta \approx 0.28$ — i.e. semantic similarity is more important than lexical on our data, but not by as much as you might think.

> **Why it matters:** This is one of two novelty contributions. Most systems use either pure cosine or a hardcoded hybrid; learning the mix from data consistently helps.

3.4 Cross-encoder reranker

What is it? A second neural network that takes a *pair* (query, chunk) and outputs a relevance score. We use `ms-marco-MiniLM-L-6-v2`, trained on MS MARCO passage-ranking data.

Why is it used? The bi-encoder's separate encoding is approximate. The cross-encoder's joint view is much more accurate at telling near-matches apart.

What role does it play? It re-ranks the top-20 from FAISS down to the top-5 that will actually go into the generator's prompt.

Why is it important? It is the single biggest retrieval-precision lever in the system. Removing it costs −0.04 grounding in our ablation.

3.5 Flan-T5-large generator

What is it? A 780-million-parameter text-to-text transformer pre-trained by Google on a wide range of instruction-following tasks.

Why is it used? It is the component that actually writes the answer in English, given the retrieved context.

What role does it play? It turns the top-5 reranked chunks into one coherent, citation-prefixed answer.

Why is it important? Without it we'd only have raw chunk text. The generator is what makes the system feel like a chatbot rather than a search engine.

We load it in `float16` (half-precision) to halve VRAM usage with negligible quality loss.

3.6 DoRA adapters + DPO training

What is it? DoRA (Weight-Decomposed Low-Rank Adaptation) is a parameter-efficient fine-tuning method. Instead of updating all 780M parameters, we freeze them and add a tiny "adapter" (∼3M parameters) that learns task-specific adjustments. DPO (Direct Preference Optimization) is a way to train the adapter from *preference pairs* ("this answer is better than that answer").

Why is it used? We want the generator to *prefer* citation-prefixed answers (`[doc_id: ...]` ...). We build preference pairs by running the base model twice — once with a citation-priming prompt (the "chosen" answer) and once without (the "rejected" answer). DPO then teaches the adapter to lean towards the chosen style.

What role does it play? Preference alignment — Component D of the "at least 3 of A–E" rubric.

Why is it important? Citations are the thing that lets the user verify the answer. Without them the whole grounding story falls apart.

3.7 CGRA information-density gate

What is it? A composite confidence score (coverage + density + specificity) that tells us how sure we are about the retrieved chunks.

Why is it used? To decide whether to (a) trust the KB, (b) augment with the web scraper, or (c) escalate right away.

What role does it play? It is the *decision maker* that routes queries to one of those three outcomes.

Why is it important? This is the second novelty contribution. It is also the component that, together with the grounding re-check, raises CER from 0.65 to 0.95. Ablating it costs −0.22 CER — the single biggest lever for the main metric.

3.8 Web-scraper fallback

What is it? A tool that fetches public web pages when the KB is thin, with a small local cache of canned answers for common queries.

Why is it used? As a second source of evidence when the KB is insufficient but the query is still answerable in principle.

What role does it play? Fallback augmentation, triggered by the CGRA gate.

Why is it important? It turns "low-confidence escalation" into "second-chance retrieval". Not every off-topic query actually needs a human.

3.9 Grounding re-check

What is it? A post-generation test that measures what fraction of the answer's important words actually appear in the retrieved context.

Why is it used? Even with all the above in place, the generator can still drift. This last check catches drift at the very end, in front of the user.

What role does it play? Final safety net. Below threshold → discard the answer, open a ticket.

Why is it important? It is the difference between "RAG that usually works" and "RAG that fails safely". Ablating it costs −0.30 CER.

4. Alternatives Analysis

For each major choice, we considered several options. This section explains what was on the table and why we picked what we did.

4.1 For the encoder

Alternatives we considered **Options considered:**

- `all-MiniLM-L6-v2` (22M params, 384-dim) **[chosen]**
- `all-mpnet-base-v2` (110M params, 768-dim, ∼5% better quality)
- OpenAI `text-embedding-3-small` (API, paid)
- `BGE-large-en` (335M params, ∼10% better, needs GPU)

Why MiniLM: it runs on CPU, fits in RAM, and has 95% of the quality of the heavier open-source alternatives. For a teaching/demo project, the quality gap is not worth the compute. For a production system we would move to BGE-large or MPNet.

4.2 For retrieval: dense vs sparse vs hybrid

Alternatives we considered **Options considered:**

- Pure dense (cosine-only)
- Pure sparse (BM25-only)
- Fixed hybrid (dense + sparse, weights guessed)
- **Learnable hybrid** (weights learned from data) **[chosen]**

Why learnable hybrid: fixed hybrids work but the right weights vary by corpus. Learning them from training triplets is cheap (two scalar parameters) and reliably better than guessing. BM25 is a natural upgrade from Jaccard for future work; we chose Jaccard for v1 because it has no tuning parameters.

4.3 For reranking

Alternatives we considered **Options considered:**

- No reranker (skip the step, use the bi-encoder's ordering)
- **Cross-encoder (ms-marco-MiniLM) [chosen]**
- ColBERT-style late interaction (more accurate, much heavier)
- LLM-as-judge reranker (use the main LLM to score pairs)

Why cross-encoder: ColBERT needs its own index. LLM-as-judge is slow and expensive per pair. The cross-encoder hits the sweet spot — about 3–5× better than no reranker, fast enough to run on 20 candidates.

4.4 For the generator

Alternatives we considered **Options considered:**

- Flan-T5-large (780M, seq2seq) **[chosen]**
- Llama-3-8B-Instruct (stronger, decoder-only)
- GPT-3.5 / GPT-4 via API (best quality, paid)
- Mistral-7B-Instruct (decoder-only, Apache 2.0)

Why Flan-T5-large: it fits on a single T4 GPU in fp16 (roughly 1.5 GB), runs at ∼1 second

per answer, and is already good at instruction-following. The seq2seq architecture is a natural fit for "given this context, produce this answer". A future upgrade to Llama-3-8B would require minimal changes to the pipeline.

4.5 For fine-tuning

Alternatives we considered **Options considered:**

- Full-parameter fine-tuning (best quality, needs ∼16 GB VRAM)
- LoRA (low-rank adapters, ∼2 GB VRAM)
- **DoRA** (decomposed LoRA, marginally better than LoRA) **[chosen]**
- Prefix tuning (lighter but weaker)

Why DoRA: the rubric explicitly recommended it. In practice DoRA's quality is very close to LoRA on a task this small, but it gives us a defensible novelty angle over vanilla LoRA. The trainable parameter count is tiny (∼0.4% of the base model), which makes the training loop fast.

4.6 For preference alignment

Alternatives we considered **Options considered:**

- Supervised fine-tuning on gold answers (needs labelled answers)
- PPO / RLHF (classic, complex, unstable)
- **DPO** (Direct Preference Optimization, closed-form) **[chosen]**
- KTO (newer, simpler than DPO, but less tested)

Why DPO: it replaces the whole RLHF pipeline with a single closed-form loss. No reward model, no RL loop. For our preference rubric ("prefer citation-prefixed answers") that is exactly the right level of machinery.

4.7 For routing (tool selection)

Alternatives we considered **Options considered:**

- LLM-based router (ask Flan-T5 to output the tool name)
- **Keyword router** (default, deterministic) **[chosen]**
- **Learned router** (MiniLM + logistic regression head, opt-in) **[chosen as alternative]**

Why keyword (default) + learned (opt-in): the LLM-based router adds an extra generation per query and is prone to feedback loops when we fine-tune the generator. The keyword router is fast and predictable. When paraphrases matter, we offer the learned router as a config flag — it is a real trained model (see `training/router.py`) that actually learns from labelled examples.

5. Code Snippets, Explained

Below are the handful of pieces of code that together define the system. Each one has a plain-English summary of what it does.

5.1 The learnable fusion training loop

This is the heart of the learnable hybrid retriever. We learn two numbers α, β that say how much to trust semantic versus lexical similarity.

```
for epoch in range(n_epochs):
    for (q, pos, neg) in triplets:
        # current weights via softmax
        a, b = softmax2(l_alpha, l_beta)

        # score positive chunk vs negative chunk
        score_pos = a * cos_pos + b * lex_pos
        score_neg = a * cos_neg + b * lex_neg
        margin    = score_pos - score_neg   # want > 0
        p         = sigmoid(margin)
        loss      = -log(p)

        # hand-derived gradients
        dL_da = (p-1) * (cos_pos - cos_neg)
        dL_db = (p-1) * (lex_pos - lex_neg)
        ...   # chain through softmax Jacobian
    # momentum update
    v_alpha = mu * v_alpha - lr * grad_alpha
    l_alpha += v_alpha
```

In plain English: for each training example we have a "positive" chunk (the right answer) and a "negative" chunk (wrong answer). We want the positive to score higher than the negative. If it doesn't, we nudge the weights in the direction that would have made it score higher. Repeat 80 times. The word "momentum" just means we remember the last nudge and keep pushing in the same direction — it helps us avoid zig-zagging.

Plain-English summary We are not using PyTorch's autograd here. Every gradient is written by hand. This makes the whole thing easier to understand and removes a heavy dependency for a component that only has two parameters.

5.2 The keyword router

```
TICKET_KEYWORDS = ['cannot log', 'login', 'password', 'broken', 'error', ...]
POLICY_KEYWORDS = ['policy', 'section', 'regulation', 'rule']

def select_tool(query: str) -> str:
    q = query.lower()
    if any(k in q for k in TICKET_KEYWORDS): return 'CreateTicket'
    if any(k in q for k in POLICY_KEYWORDS): return 'GetPolicy'
    return 'SearchKB'
```

In plain English: look at the question in lowercase. If it contains any trouble-words, treat it as a ticket. If it mentions the policy jargon, look it up. Otherwise search the KB. No neural network, no LLM call — just string matching. This takes < 1 ms.

5.3 The learned router

```
class LearnedRouter:
```

```
    CLASSES = ('SearchKB', 'GetPolicy', 'CreateTicket')

    def fit(self, queries, labels, epochs=200, lr=0.5):
        X = self.encoder.encode(queries)            # (N, 384)
        y = one_hot(labels)                         # (N, 3)
        W = random_init(384, 3)
        for _ in range(epochs):
            logits = X @ W + b
            probs  = softmax(logits)
            grad   = X.T @ (probs - y) / N          # cross-entropy gradient
            W      -= lr * grad
        self.W = W

    def predict(self, query):
        x = self.encoder.encode([query])[0]
        return CLASSES[argmax(x @ self.W + self.b)]
```

In plain English: a classifier trained on ∼35 labelled examples. We encode each question as a 384-dim vector, then learn a linear map from that vector to one of three classes. This catches paraphrases the keyword router misses — for example, "stuck on the sign-in page" which has no keyword in our list but whose *meaning* is clearly a ticket.

5.4 The CGRA information-density score

```
def information_density(query, chunks, a=0.4, b=0.4, g=0.2):
    if not chunks: return 0.0
    # 1. Coverage: fraction of query terms present in retrieved chunks
    q_terms = tokens(query) - STOPWORDS
    all_terms = union(tokens(c) for c in chunks)
    coverage = len(q_terms & all_terms) / len(q_terms)

    # 2. Density: avg cross-encoder score (sigmoid-squashed)
    density = mean(sigmoid(c['ce_score']) for c in chunks)

    # 3. Specificity: chunks with consistent lengths score higher
    specificity = 1 - min(stdev(lengths) / 150, 1.0)

    return a*coverage + b*density + g*specificity
```

In plain English: three questions — are the query's important words in what we retrieved? Do the chunks score high with the reranker? Are they uniform (or is the retrieval noisy)? Blend with weights $(0.4, 0.4, 0.2)$.

5.5 The post-generation grounding check

```
def grounding_score(answer: str, context: str) -> float:
    a = tokens(answer) - STOPWORDS
    c = tokens(context) - STOPWORDS
    if not a: return 0.0
    return len(a & c) / len(a)

# After generation:
gs = grounding_score(answer, context)
if gs < 0.08:
    # answer did not come from the context -- escalate
    open_ticket(...)
```

In plain English: take the answer the LLM produced, remove stopwords, and check what fraction of the remaining words actually appeared in the retrieved context. If it is below 8%, the LLM is probably making things up — throw the answer away and open a ticket.

5.6 The tool-call contract (pydantic)

```
class SearchKBArgs(BaseModel):
    query: str = Field(..., min_length=1)
    top_k: int = Field(5, ge=1, le=50)

class CreateTicketArgs(BaseModel):
    summary:  str = Field(..., min_length=1)
    category: Literal['account', 'payments', 'general'] = 'general'
    severity: Literal['low', 'medium', 'high']          = 'medium'

class ToolCall(BaseModel):
    tool: Literal['SearchKB', 'GetPolicy', 'CreateTicket']
    arguments: dict     # cross-validated by field_validator
```

In plain English: every tool call is validated before execution. Invalid category? Error before anything runs. Missing query? Error before anything runs. This is what "structured tool outputs" means in practice.

6. End-to-End Story: One Query's Journey

Let's follow a single query all the way through the pipeline.

User asks: "How do I appeal a denied disability claim?"

6.1 Stage 1 — Router

The query enters `select_tool()`. It lowercases to "how do i appeal a denied disability claim?". No "login" / "broken" / "error" keywords, so not a ticket. No "section" / "policy" keywords, so not a policy lookup. Default: **SearchKB**.

6.2 Stage 2 — Build arguments

Because the query contains "how do" (a synthesis keyword), we expand `top_k` from 5 to 8. The agent constructs:

```
{"tool": "SearchKB", "arguments": {"query": "...", "top_k": 8}}
```

This gets validated against `SearchKBArgs` — query is non-empty, top_k is in bounds. OK.

6.3 Stage 3 — FAISS retrieval with hybrid fusion

The fine-tuned MiniLM encodes the query into a 384-dim vector. FAISS returns the 32 nearest chunk vectors. For each, we compute both a cosine score and a Jaccard score, blend with $\alpha = 0.72, \beta = 0.28$, and keep the top 8.

6.4 Stage 4 — Cross-encoder reranking

We pass the 8 (query, chunk) pairs through `ms-marco-MiniLM`. Each gets a `ce_score` attached. Sort by `ce_score`, keep top 5.

6.5 Stage 5 — CGRA gate

We compute ID over those 5 chunks:

- coverage = 0.83 (most query terms appear in the retrieved text)
- density = 0.79 (high cross-encoder scores)
- specificity = 0.91 (chunks are about the same length)

$\text{ID} = 0.4 \cdot 0.83 + 0.4 \cdot 0.79 + 0.2 \cdot 0.91 = 0.832$.

That is well above $\tau = 0.45$, so the KB is sufficient — we do NOT call the WebScraper.

6.6 Stage 6 — Generation

We build the prompt (top-5 chunks as context) and call Flan-T5 (with DoRA adapters attached). Output:

> *"[doc_id: appeal_process_2019] If your disability claim is denied, you have 60 days to file a request for reconsideration. If the reconsideration is also denied, you may request a hearing before an Administrative Law Judge. Further appeals go to the Appeals Council and then Federal Court."*

6.7 Stage 7 — Grounding check

Tokens in the answer (minus stopwords): {denied, claim, 60, days, file, request, reconsideration, hearing, administrative, law, judge, appeals, council, federal, court, disability}.

Of those 16 tokens, 14 appear in the retrieved context. Grounding = 0.875, well above $\theta = 0.08$.

6.8 Stage 8 — Return to user

The `AgentResponse` object is:

```
{
  "final_answer":     "[doc_id: appeal_process_2019] If your...",
  "citations":        ["appeal_process_2019__chunk_3", ...],
  "grounding_score": 0.875,
  "id_score":         0.832,
  "ticket":           null,
  "tool_trace":       [{"step": 1, "tool": "SearchKB", ...}],
  "latency_ms":       1247.3
}
```

The user sees the answer plus the citation. If they want to verify, they click the citation and read the original passage.

6.9 What happens if something goes wrong?

Bad case 1 — Sparse KB

User asks about something the KB does not cover well. CGRA's ID is $0.31 < 0.45$. The web scraper runs. Returns 2 chunks from a cached entry. Combined with the KB chunks, ID rises to 0.52. We proceed to generation.

Bad case 2 — KB and web both thin

Even after web scraping, ID is still $0.15 < \tau_{\min} = 0.20$. We give up and open a ticket with category=general, severity=medium. The user sees a ticket number; a human follows up.

Bad case 3 — LLM hallucinates despite good retrieval

Good KB, high ID, generation runs. But the LLM produces an answer that barely overlaps the context (grounding = 0.05). The post-generation check catches it. We open a ticket with severity=high ("ungrounded answer") and tell the user their question will be reviewed.

Plain-English summary The system has **three separate escalation paths** — keyword routing, CGRA gate failure, and grounding-check failure — and each one exists to catch a different failure mode. Together they make sure the system is conservative: it would rather open a ticket than ship a bad answer.

7. Overfitting, Underfitting, and Leakage Checks

A legitimate question: how do we know the system actually *generalises* rather than just memorising the training documents? The project ships a diagnostics module (`training/diagnostics.py`) that runs five checks on every evaluation.

7.1 Document-level train/test split

> Plain-English summary: The 80/20 split happens at the document level, not the chunk level. Every test chunk comes from a document the retriever has never embedded during training.

Why this matters: chunks from the same document often share phrasing. If you split by chunk, you end up with situations where the training and test sets contain near-duplicates of the same paragraph. Retrieval metrics then look artificially high because the model has, in effect, seen the test data. Splitting at the document level prevents this.

The diagnostics cell reports `train/test docs disjoint: yes`. If it ever says `no`, the split is broken and the numbers cannot be trusted.

7.2 Retriever train-vs-test probe

> Plain-English summary: If train P@5 is much higher than test P@5, the retriever has memorised training chunks. If both are low, it hasn't learned anything.

The diagnostic picks 40 random chunks from each split, builds a query from the first 10 words of each chunk, and measures P@5 and MRR. Then it prints both numbers and a verdict:

- **OK (test $\geq$ train)**: the retriever generalises
- **mild overfit**: train is a little ahead of test; within noise
- **OVERFIT ($\Delta > 0.25$)**: retriever memorised; lower the LR or fewer fine-tune epochs
- **UNDERFIT (both low)**: retriever isn't learning; check that fine-tuning ran end-to-end

7.3 Fusion-weight sanity check

The learned α and β should not collapse to 1.0 / 0.0 (one signal dominating) and should not stick at 0.5 / 0.5 (no differentiation). A healthy run typically lands near $\alpha \approx 0.7, \beta \approx 0.3$.

7.4 DoRA early stopping

> Plain-English summary: DoRA trains 2.5 million parameters on 10–40 preference pairs. That's a lot of parameters per example. Without early stopping it will overfit.

The training loop holds out 20% of pairs as a validation set and stops as soon as validation loss stops improving for two consecutive epochs. Typical runs stop at epoch 2 or 3 out of a configured 3, confirming the guard is active.

7.5 Router generalisation

The learned router (opt-in, in `training/router.py`) is trained on 35 labelled intents. The diagnostic evaluates it on 8 held-out paraphrases it has never seen:

- "I cannot access my account at all" → expected CreateTicket
- "Show me the official policy on SSI" → expected GetPolicy
- "When do survivor benefits kick in?" → expected SearchKB

A healthy router gets 7/8 or 8/8 correct. Below 6/8 is a warning sign.

7.6 Cache-leakage scan

The WEB_CACHE is a six-entry response-time cache for common web queries. The diagnostic scans every evaluation query to check whether any of them is a literal match for a cache key (which would be cheating). It's fine if a query *shares keywords* with a cache key — that's the cache doing its job — but the query must not be the key verbatim.

8. Testing the Chatbot: 30 Curated Demo Questions

The project ships 30 hand-written demo questions in `training/demo_questions.py`, split into four categories so you can see how each layer of the system behaves:

8.1 Standard questions (8)

Questions whose answer is definitely in the KB. Expected behaviour: grounded answer with citation, no ticket.

Examples:

- "What documents do I need to apply for Social Security retirement benefits?"
- "How many work credits do I need to qualify for Social Security?"
- "At what age should I enroll in Medicare Part A and Part B?"

8.2 Paraphrased questions (8)

Same intents as the standard set, but worded colloquially. This is where the learned router shines and where a purely keyword-based system would miss.

Examples:

- "I'm turning 62 next year — when's the earliest I can start getting checks?"
- "My husband passed away last month. Am I entitled to anything?"
- "I work part-time and collect Social Security. Is there a limit on my earnings?"

8.3 Account-issue questions (7)

Should skip search entirely and route to CreateTicket via the keyword router.

Examples:

- "I can't log into my account"
- "My online portal is throwing an error when I sign in"
- "I am locked out of my Social Security account"

8.4 Out-of-scope questions (7)

Genuinely not in the KB. The CGRA gate should fire; the system should either augment with the web scraper or escalate via a low-confidence ticket.

Examples:

- "What is the tax treaty between the US and France for expatriates?"
- "What's the weather forecast for my local SSA office tomorrow?"
- "Can I invest my Social Security contributions in cryptocurrency?"

8.5 Expected behaviour summary

Category	Expected tool	Grounding	Ticket
STANDARD	SearchKB	> 0.5	No
PARAPHRASED	SearchKB or learned	> 0.5	No
ACCOUNT_ISSUE	CreateTicket	0.0	Yes
OUT_OF_SCOPE	SearchKB $\rightarrow$ escalate	varies	Mostly

Run this block in `notebook/colab.ipynb` to see all 30 in action:

```
from training.demo_questions import all_demo_questions
for item in all_demo_questions():
    r = run_agent(item['query'], ...)
    print(item['category'], item['query'])
    print('  ->', r.final_answer[:120])
```

9. Quantization and Performance

Plain-English summary Every model in the pipeline — generator, bi-encoder, cross-encoder — is loaded through one central helper (`models/quantization.py`) so precision is consistent everywhere. On a T4 GPU this halves total VRAM and speeds inference up by roughly 1.4×.

9.1 What is quantization?

A neural network's weights are normally stored as 32-bit floating-point numbers (fp32). Quantization replaces that with a smaller format:

- **float16 (half precision)** — 16 bits per number. Halves memory, roughly 1.4× faster on modern GPUs. No measurable quality drop on our metrics.
- **int8** — 8 bits per number, via the `bitsandbytes` library. Quarters memory but needs a GPU and the extra library; we support it but don't use it by default.

9.2 How we apply it

Every config file has a `dtype` key for each model:

```
embeddings:  { dtype: "float16" }    # bi-encoder
reranker:    { dtype: "float16" }    # cross-encoder
generator:   { dtype: "float16" }    # Flan-T5
```

At load time, `models/quantization.py::resolve_dtype()` translates each string into a `torch.dtype`. If the system has no GPU, `float16` is auto-downgraded to `float32` with a warning — half precision is actually slower than full precision on most CPUs.

9.3 What we measured

Model	fp32 VRAM	fp16 VRAM
Flan-T5-large	3.1 GB	1.5 GB
Cross-encoder (MiniLM)	0.35 GB	0.18 GB
Bi-encoder (MiniLM)	0.22 GB	0.11 GB
Total inference footprint	3.67 GB	1.79 GB

fp16 cuts the total from about 3.7 GB to 1.8 GB, which is what lets the whole pipeline run on the free Colab tier.

9.4 Why not always use int8?

Int8 saves more memory but has three practical issues on our workload:

- `bitsandbytes` is CUDA-only — broken on CPU Colab runtimes
- `sentence-transformers` and `CrossEncoder` don't integrate cleanly with int8 quantization
- The Flan-T5 fp16 footprint is already comfortable on T4 (1.5 GB)

We kept the int8 code path in the helper so users with very tight budgets can opt in with a one-line config change; by default everything runs in fp16 on GPU and fp32 on CPU.

10. Conversation Memory

Plain-English summary: A support chatbot that can't remember what you just asked is annoying. We added a small bounded buffer that keeps the last four turns per session and uses them to (a) resolve follow-up references like "and what about the early option?" and (b) let the generator see the dialogue flow.

10.1 Why we need it

A stateless system works fine for one-shot queries ("what's the retirement age?") but breaks on the very first follow-up. The user asks "what's full retirement age?", gets 67, then asks "and early?". Without memory, the agent retrieves on the literal string "and early?"—which matches nothing in the knowledge base—and opens a ticket instead of answering.

10.2 Design choices

Three deliberate decisions:

- **Short-term only.** Memory lives in RAM for the duration of one session. Nothing is written to disk. Persistent memory on a support system is a privacy/correctness minefield; the right pattern is to log interactions, review them, and retrain on curated examples, not absorb them live.
- **Bounded buffer (last K=4 turns).** Flan-T5 has a 1024-token context window; unbounded history blows the budget. Four turns is the empirical sweet spot: enough context for follow-ups, never more than ~800 characters of history in the prompt.
- **Influences inference only, not training.** No gradient flows through memory. The system is still offline-trained; memory is a runtime-only feature.

10.3 How memory affects retrieval

We use a simple *reference resolution* heuristic: if a query is short (<15 tokens) AND contains a reference word ("it", "that", "earlier", "and", "what about"), prepend the most-recent user turn before retrieving. Otherwise pass through untouched.

```
m = ConversationMemory(max_turns=4)
m.add_turn("What is full retirement age?", "67.")

m.resolve_reference("And the earlier option?")
# -> "What is full retirement age? And the earlier option?"
# (memory-expanded query used for retrieval; LLM still sees the
#  original short query plus the history block.)
```

This is intentionally conservative—we'd rather miss a resolvable follow-up than inject noise into a standalone query.

10.4 How memory affects generation

The generator's prompt grows a preamble:

```
You are a customer support agent.
Answer ONLY using the context. Start with [doc_id: ...].

Previous conversation:
User: What is full retirement age?
Assistant: [doc_id: retirement_age_doc_1] Full retirement age is 67.
```

```
User: And the earlier option?

Context: <retrieved chunks>
Question: And the earlier option?
Answer [cite first]:
```

The assistant sees who said what, so it interprets "earlier" in light of the previous turn. Each recorded assistant turn is clipped at 200 characters so long answers don't dominate the budget.

10.5 Where it's wired

- Single source of truth: `training/memory.py`
- Each session owns one `ConversationMemory` instance
- `demo/cli.py`: one memory per CLI invocation, cleared via `/reset`
- `demo/app.py`: per-`session_id` memory; `POST /session/{id}/reset` endpoint clears it
- `run_agent(..., memory=mem)` does the rest: reference resolution, history prefix, per-turn recording (at every response path, including escalation)

11. Limitations and Scope

11.1 Offline-trained, inference-only at runtime

Plain-English summary This copilot does not learn continuously. It is trained once through four fine-tuning stages, then serves read-only inference. No live weight updates, no feedback absorption, no online adaptation.

During deployment, no user interaction ever modifies a model weight. No telemetry is collected for incremental learning. If the KB or user patterns change, you must rerun `make train` on updated labelled data.

This is deliberate, not a limitation of the code. A support system that updated itself from live traffic would be hard to audit, prone to feedback loops, and vulnerable to absorbing user-supplied "corrections" that are actually wrong. The correct production pattern is: log interactions, have a human curator review them, periodically retrain on vetted examples.

11.2 Grounding check is lexical, not semantic

The post-generation grounding score is the fraction of the answer's non-stop-word tokens that also appear in the retrieved context. Useful heuristic, not a guarantee of factual correctness.

What it catches reliably: invented statistics/names/numbers not in context, drift off-topic from retrieved evidence, hallucination using entirely different vocabulary.

What it does NOT catch: answers that copy many words but misinterpret them (high overlap, wrong meaning), correctly paraphrased answers using different wording (low overlap, penalised unfairly), logical/arithmetic errors, subtle factual slips where only one number is wrong.

A stronger check would use Natural Language Inference: feed `(answer, context)` into an entailment classifier like `cross-encoder/nli-deberta-v3-base`. It would be a drop-in replacement — see `agent.py::_grounding` for the call site.

11.3 Other limitations

- **Evaluation set size.** The 20-query headline metric set is too small for statistical significance. Deltas are directional, not definitive. A 200-query stratified held-out set with bootstrap CIs is the natural next step.
- **DoRA self-generated preferences.** DPO pairs are built by running the base model twice with different prompts — a known failure mode (distributional collapse). We mitigate with 20% val hold-out + early stopping, but don't stress-test longer training.
- **Policy DB is a small topic-map.** The 5-entry `policy_db.json` matches this dataset. A real deployment would need a semantic-retrieval index over the policy documents.

12. Where to Look in the Code

Path	What lives there
`config/config.yaml`	Central hyperparameters (chunk size, LR, dtypes, etc.)
`config/baseline.yaml`	Baseline config (no tuning, no CGRA, no grounding check)
`config/full.yaml`	Full config (everything on)
`data/raw/policy_db.json`	Static policy sections (editable JSON)
`models/quantization.py`	Central precision helper (fp16/int8 auto-select)
`models/base_model.py`	Flan-T5 loader, strict DoRA adapter attach
`models/embeddings.py`	Bi-encoder loader + MNRL fine-tuner
`retrieval/retriever.py`	FAISS + learnable α, β fusion
`retrieval/reranker.py`	Cross-encoder reranker
`training/tools.py`	Three tools + pydantic dispatcher + web scraper
`training/agent.py`	CGRA gate + router + policy-mapper + main agent loop
`training/router.py`	Learned logistic-regression router (opt-in)
`training/memory.py`	Session memory (bounded buffer)
`training/diagnostics.py`	Overfit/underfit/leakage checks
`training/demo_questions.py`	30 curated test questions (4 categories)
`training/test_questions_v2.py`	30 v2 test questions (6 categories incl. multi-turn)
`training/train.py`	CLI: `-stage biencoder|fusion|dora|router|all`
`training/evaluation.py`	CLI: `-config ...|-compare`
`utils/metrics.py`	P@K, R@K, MRR, BLEU, ROUGE-L, grounding, CER
`utils/schema.py`	Pydantic `ToolCall` + `AgentResponse`
`demo/app.py`	FastAPI + session memory endpoints
`demo/cli.py`	Interactive CLI with `/reset` and `/memory` commands
`notebook/colab.ipynb`	Google Colab quickstart
`tests/`	Pytest suite (35 tests, offline, <15s)

End of beginner's documentation. See `report_v2.pdf` *for the formal 5-page technical paper.*